Agenda

01 What is a REST API?

02 Why REST API?

03 Data Exchange Formats

04 JSON vs. XML

05 Application Architecture

06 Monolith Architecture

07 Why Monolith Architecture

08 Microservice Architecture

09 Why Microservice Architecture

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

What is a REST API?

Rest API or Restful API is an architectural style that defines a set of constraints while building a Web API to make it easier to use and have a more predictable structure

METHOD	URL	DESCRIPTION
GET	/users	GET ALL USERS
POST	/users	SAVE A USER
GET	/users/:id	GET A USER WITH PARTICULAR ID
PUT|PATCH	/users/:id	UPDATE A USER WITH PARTICULAR ID
DELETE	/users/:id	DELETE A USER WITH PARTICULAR ID

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why REST API?

Rest API provides several benefits

- Client–Server
- Reliability
- Scalability
- Language Independence

Allows developers to separate client and server code which helps is system decoupling

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why REST API?

Rest API provides several benefits

- Client–Server
- Reliability
- Scalability
- Language Independence

Separation of client and server systems has the advantage of evolving the systems separately

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why REST API?

Rest API provides several benefits

- Client–Server
- Reliability
- Scalability
- Language Independence

Since client and server are separate they can be scaled separately and easily

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why REST API?

Rest API provides several benefits

- Client–Server
- Reliability
- Scalability
- Language Independence

REST API is based on a uniform interface which mean if backend is moved from one language to other the client will not change

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Data Exchange Formats

When building a REST API one of the most important decisions to make is data exchange formats

There can be multiple formats which an api can support or choose to support one of the multiple formats available

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Data Exchange Formats

There are three types of Data Exchange Formats available:

- JSON
- XML
- YAML

JSON is simple and widely supported format for data exchange over the internet

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Data Exchange Formats

There are three types of Data Exchange Formats available:

- JSON
- XML
- YAML

XML was once the most popular data exchange formats once but it was very verbose and difficult to parse

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Data Exchange Formats

There are three types of Data Exchange Formats available:

JSON

XML

YAML

YAML is the newest data exchange format, it's easy to read and parse, but not the most popular

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

JSON vs. XML

JSON and XML are the most popular formats for rest api's to exchange data

Although both JSON and XML are quite popular but they both have their own advantages and disadvantages

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

JSON vs. XML

XML	JSON
More complicated than JSON	Simple to read and write
Verbose	Compact
Document-oriented	Data-oriented
Doesn't support arrays	Supports arrays
HTML-like syntax	JavaScript-like syntax

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Application Architecture

Application Architecture is about structuring different services or features in applications so that they interact with each other in the most efficient way

There are several ways to architect your apps. Two of the most popular ways are Monolith and Microservice architecture

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Monolith Architecture

Monolith Architecture is used when all the services are grouped in a single application packages

All the services in the packages are all compiled, run and deployed together

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why Monolith Architecture?

There are several advantages of Monolith Architecture

- Development
- Testing
- Deployment
- Scalability

Development of applications is easy as all the services are centralized in a single package

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why Monolith Architecture?

There are several advantages of Monolith Architecture

- Development
- Testing
- Deployment
- Scalability

Testing of applications is also very easy as end to end tests can be run very easily with all services running

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why Monolith Architecture?

There are several advantages of Monolith Architecture

- Development
- Testing
- Deployment
- Scalability

Deployment of the entire package is easier as it simply requires us to build the package and move it to the server

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

🔄 Monolith vs Microservices

Monolith	Microservices
One app	Many small services
Simple	Complex
Hard to scale	Easy to scale

🎯 Real-life analogy

👉 Monolithic = **one big restaurant kitchen**
👉 Microservices = **multiple specialized kitchens**

Microservice Architecture

Microservice Architecture is used by separating services into their own separate packages so that they can be changes independently of each other

In Microservice Architecture these services are separated from each other but can communicate with each other over an agreed upon protocol like REST, RPC etc.

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why Microservice Architecture?

There are several advantages of Microservice Architecture

- Decoupled
- Independence
- Ease of Change
- Deployment
- Scalability

Applications are decomposed into several decoupled services which interact with each other

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

REST vs. RPC: Key Differences

Feature	REST (Representational State Transfer)	RPC (Remote Procedure Call)
Core Concept	**Resource-based**: Focuses on "nouns" (things).	**Action-based**: Focuses on "verbs" (functions).
URL Example	`/users/123`	`/getUserById`
HTTP Methods	Uses standard verbs: `GET`, `POST`, `PUT`, `DELETE`.	Usually just uses `POST` for everything.
Structure	Strict, standardized architectural style.	Flexible; defined by the specific function.
Coupling	**Loose**: Client and server are less dependent.	**Tight**: Client often needs to know the exact method.
Readability	**High**: Predictable and self-descriptive.	**Medium**: Depends on how methods are named.
Best Use Case	Public APIs, CRUD apps, web services.	Internal microservices, high-performance tasks (gRPC).

Why Microservice Architecture?

There are several advantages of Microservice Architecture

- Decoupled
- Independence
- Ease of Change
- Deployment
- Scalability

Each service can be independently developed with different teams

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why Microservice Architecture?

There are several advantages of Microservice Architecture

- Decoupled
- Independence
- Ease of Change
- Deployment
- Scalability

Each service could be changed and implemented in new technologies without any issues

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why Microservice Architecture?

There are several advantages of Microservice Architecture

- Decoupled
- Independence
- Ease of Change
- Deployment
- Scalability

Microservice architecture enables each microservice to be deployed independently

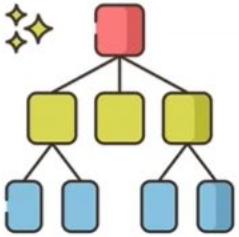
© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why Microservice Architecture?

There are several advantages of Microservice Architecture

- Decoupled
- Independence
- Ease of Change
- Deployment
- Scalability

Microservice architecture enables each service to be scaled independently



© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.